
UNIVERSIDADE FEDERAL DE SÃO CARLOS

Centro de Ciências Exatas e Tecnologia

Departamento de Computação

Laboratório de Arquiteturas de Alto Desempenho

Prática 7 — Arquitetura Mesh 4×4 para
Processamento Digital de Imagens com Roteamento XY

Professor: *Dr. Emerson Carlos Pedrino*

Pedro Augusto Faria (RA: 821124)

Curso: *Engenharia da Computação*

1 Introdução

As redes intra-chip (*Network-on-Chip*, NoC) representam um paradigma arquitetural fundamental para sistemas de processamento paralelo de alto desempenho implementados em hardware reconfigurável. Em substituição aos barramentos compartilhados tradicionais, as NoCs oferecem comunicação escalável, de alta largura de banda e baixa latência entre múltiplos núcleos de processamento (*Processing Elements*, PEs) dispostos em topologias regulares — sendo a malha bidimensional (*mesh*) a mais empregada na literatura e na indústria.

Nesta prática, implementa-se uma rede *mesh* 4×4 em FPGA para o processamento digital de imagens (PDI), especificamente para a aplicação do filtro *sal e pimenta* (filtro de máximo e mínimo sobre janela de 3×3 pixels). A rede é composta por 16 nós (*mesh_node*), cada um dotado de um núcleo de processamento (*modulo_sal_e_pimenta*) e de um roteador com política de roteamento determinístico XY. Os dados de imagem provenientes de uma ROM são injetados no nó de entrada, roteados pela rede conforme um caminho configurável de até quatro saltos (*hops*), e a saída do nó final é exibida em monitor VGA de resolução 640×480 .

O roteamento XY (*dimension-order routing*) garante ausência de *deadlock* na rede ao impor que o pacote se desloca primeiro na direção horizontal (eixo *x*, colunas) até alinhar-se com a coluna destino, e somente então na direção vertical (eixo *y*, linhas). Esta política é implementada localmente em cada nó, sem necessidade de tabelas de roteamento globais, tornando-a escalável e sintetizável de forma compacta em FPGA.

O módulo controlador *noc_controller* define estaticamente os endereços e os modos de operação (máximo ou mínimo) dos quatro nós do caminho de processamento ativo, permitindo a experimentação rápida de diferentes configurações de roteamento — coluna vertical, grupo no topo, ou diagonal — simplesmente alternando atribuições de constantes no código.

1.1 Fundamentação Teórica

1.1.1 Redes *Mesh* e Roteamento XY

Uma rede *mesh* $M \times N$ é um grafo regular em que cada nó (*col*, *row*) está conectado a até quatro vizinhos: norte (*col*, *row* − 1), sul (*col*, *row* + 1), leste (*col* + 1, *row*) e oeste (*col* − 1, *row*). Os nós das bordas possuem menos conexões (sem vizinho externo). A topologia é atraente por sua regularidade, que facilita o roteamento e o leiaute físico.

O roteamento XY é um algoritmo de roteamento determinístico e livre de *deadlock* para redes *mesh* bidimensionais. Dado um nó fonte (c_s, r_s) e um nó destino (c_d, r_d), o caminho percorrido é:

1. Mover na direção horizontal (leste se $c_d > c_s$, oeste se $c_d < c_s$) até que $c_{atual} = c_d$;

2. Mover na direção vertical (sul se $r_d > r_s$, norte se $r_d < r_s$) até que $r_{atual} = r_d$.

Este ordenamento garante que não há ciclos no grafo de dependência de canais, o que é condição suficiente para ausência de *deadlock* [1].

1.1.2 Endereçamento dos Nós

Na implementação adotada, cada nó é identificado por um endereço de 4 bits $\langle col[1 : 0], row[1 : 0] \rangle$, onde os dois bits mais significativos representam a coluna e os dois menos significativos representam a linha (ambos indexados de 0 a 3). Assim, o nó na coluna 1, linha 2 tem endereço 4'b0110. A Tabela 1 lista o mapeamento completo para a *mesh* 4×4.

Tabela 1: Mapeamento de endereços para a *mesh* 4×4.

	Col 0	Col 1	Col 2	Col 3
Lin 0	0000	0100	1000	1100
Lin 1	0001	0101	1001	1101
Lin 2	0010	0110	1010	1110
Lin 3	0011	0111	1011	1111

1.1.3 Filtro Sal e Pimenta

O filtro *sal e pimenta* é um filtro de ordem estatística utilizado para a remoção de ruído impulsivo em imagens digitais. Para cada pixel (x, y) , considera-se a janela de vizinhança 3×3 centrada em (x, y) , composta por 9 pixels. O filtro de máximo substitui o pixel central pelo valor máximo da janela (produz o efeito *sal*), enquanto o filtro de mínimo substitui pelo valor mínimo (produz o efeito *pimenta*). Para imagens binárias de 1 bit por pixel, os filtros de máximo e mínimo equivalem, respectivamente, à dilatação e erosão morfológicas.

Na implementação em hardware, a janela 3×3 é construída por meio de registradores de deslocamento que modelam linhas de atraso (*line buffers*), armazenando os pixels das duas linhas anteriores à linha corrente. Dado que a imagem tem largura de 256 pixels (conforme a máscara de janela), cada linha de atraso requer 256 registradores de 1 bit.

1.1.4 Concatenador de Endereços para ROM

O módulo concatenador (VHDL) forma o endereço de 15 bits para acesso à ROM de imagem a partir das coordenadas de pixel do controlador VGA. Os 7 bits menos significativos da linha (`pixel_row[6:0]`) são concatenados com os 8 bits menos

significativos da coluna (`pixel_column[7:0]`), formando um endereço que cobre uma imagem de $128 \times 256 = 32.768$ pixels — correspondente à região de interesse definida pela máscara de janela.

1.1.5 Máscara de Janela

O módulo `mascara_janela` (VHDL) delimita a região ativa de processamento da imagem no quadro VGA: apenas pixels com `pixel_column < 256` e `pixel_row < 128` recebem o sinal `video_out = '1'`. Fora desta região, `video_out = '0'`. Este sinal de máscara é utilizado em múltiplos pontos do *top-level* para habilitar seletivamente o clock da rede *mesh* e a exibição VGA, garantindo que o processamento ocorra apenas nos pixels pertencentes à imagem.

1.2 Objetivos

- Implementar uma rede *mesh* 4×4 em Verilog com roteamento determinístico XY na FPGA Altera DE1 (Cyclone II);
- Integrar 16 núcleos de processamento `modulo_sal_e_pimenta` como nós da rede, cada um capaz de realizar filtragem de máximo ou mínimo;
- Configurar caminhos de processamento de até 4 saltos por meio do módulo `noc_controller`, experimentando diferentes topologias (coluna vertical, grupo no topo, diagonal);
- Exibir a imagem original e a imagem filtrada em monitor VGA 640×480, com seleção de modo por chaves (*switches*);
- Demonstrar o conceito de NoC aplicada a PDI, evidenciando a flexibilidade arquitetural oferecida pelo roteamento XY configurável.

1.3 Metodologia

O projeto foi desenvolvido no ambiente Quartus Prime (Versão 24.1std.0 Build 1077, Lite Edition), com módulos descritos em Verilog e VHDL, integrados por diagrama esquemático BDF. O módulo `VGA_SYNC` foi fornecido pelo professor. Os módulos foram sintetizados para a FPGA Cyclone II EP2C20F484C7 da placa Altera DE1, com pinagem definida pelo arquivo de pinos oficial.

2 Desenvolvimento Teórico

2.1 Topologia da Rede *Mesh* 4×4

A rede *mesh* 4×4 é composta por 16 nós dispostos em uma grade regular de 4 colunas por 4 linhas, totalizando 24 enlaces bidirecionais internos (12 horizontais e 12 verticais). Cada nó possui até 4 portas de comunicação (norte, sul, leste, oeste), e os nós de borda têm as portas sem vizinho conectadas a zero. A Figura 1 apresenta o diagrama BDF do *top-level*, onde a instância *mesh_4x4* concentra toda a estrutura da rede.

O fluxo de dados na rede é serial (1 bit por ciclo de clock), compatível com a natureza do sinal de vídeo VGA, que também é serial: os pixels chegam em sequência, linha por linha, ao longo do quadro. O clock da rede é derivado diretamente do pixel clock VGA, habilitado apenas durante a região ativa da imagem (controlado pelo sinal `clk2 = mask & pclk & on`), garantindo que os registradores de deslocamento internos ao `modulo_sal_e_pimenta` avancem exatamente um pixel por ciclo ativo.

2.2 Roteamento XY na Implementação

Na implementação do módulo `mesh_node`, o roteamento XY é realizado comparando a posição atual do nó (extraída do parâmetro `MY_ADDR`) com a posição do próximo nó no caminho de processamento (`next_addr`). As quatro decisões de roteamento são lógica combinacional pura:

$$\text{go_s} = \text{on_path} \wedge \neg \text{is_hop4} \wedge (\text{my_row} < \text{next_row}) \quad (1)$$

$$\text{go_n} = \text{on_path} \wedge \neg \text{is_hop4} \wedge (\text{my_row} > \text{next_row}) \quad (2)$$

$$\text{go_e} = \text{on_path} \wedge \neg \text{is_hop4} \wedge (\text{my_row} = \text{next_row}) \wedge (\text{my_col} < \text{next_col}) \quad (3)$$

$$\text{go_w} = \text{on_path} \wedge \neg \text{is_hop4} \wedge (\text{my_row} = \text{next_row}) \wedge (\text{my_col} > \text{next_col}) \quad (4)$$

Nota-se que a prioridade implícita das condições em (1)–(4) implementa o roteamento XY: a comparação de colunas (`go_e`, `go_w`) é condicionada à igualdade de linhas (`my_row = next_row`), o que significa que o pacote só se desloca horizontalmente após ter atingido a linha do destino. Contudo, como nesta implementação o deslocamento vertical é processado primeiro (a condição de linha é verificada antes da condição de coluna na cadeia de prioridade), a ordem efetiva é YX, não XY. Isto não afeta a corretude do roteamento para a topologia *mesh* simples, mas é uma nuance de implementação a considerar.

2.3 Cadeia de Processamento e Seleção de Modo

O caminho de processamento é definido por quatro endereços de salto (`hop1_addr` a `hop4_addr`) e quatro bits de modo (`hop1_max` a `hop4_max`), todos provenientes do módulo `noc_controller`. O nó `hop1` recebe diretamente o dado de entrada da ROM (`din_entry`); os nós `hop2`, `hop3` e `hop4` recebem o dado processado pelo nó anterior via roteamento.

Em cada nó do caminho, o módulo `modulo_sal_e_pimenta` processa o fluxo serial de pixels e produz dois sinais: `max_bit` (resultado do filtro de máximo) e `min_bit` (resultado do filtro de mínimo). O bit `my_max` seleciona qual dos dois é encaminhado para o próximo nó:

$$\text{pe_out} = \text{my_max} ? \text{pe_max_out} : \text{pe_min_out} \quad (5)$$

O nó `hop4` (destino final) disponibiliza `pe_out` no sinal `dout_final`, que é coletado pelo *top-level* e exibido no VGA. Os nós fora do caminho simplesmente retransmitem o dado recebido sem processá-lo.

2.4 Configurações de Roteamento Disponíveis

O módulo `noc_controller` oferece três configurações comentadas no código, selecionáveis por descomentação:

- **Coluna vertical** (nós `n10` → `n11` → `n12` → `n13`): percorre a coluna 1 de cima para baixo, com MAX no primeiro e último nó e MIN nos intermediários;
- **Grupo no topo** (nós `n00` → `n01` → `n11` → `n12`): forma um L na região superior esquerda da *mesh*;
- **Diagonal** (nós `n11` → `n12` → `n22` → `n23`): percorre dois nós verticalmente na coluna 1 e depois dois nós na coluna 2, formando uma diagonal na região central da *mesh*. Esta é a configuração ativa no código submetido.

2.5 Sincronismo e Clock Gated

O uso de *clock gating* (`clk2 = mask & pclk & on`) é uma técnica de redução de consumo de energia que desabilita o clock dos registradores de deslocamento durante os períodos de *blanking* VGA e fora da região de imagem. Em FPGA, o *clock gating* deve ser tratado com cuidado para evitar *glitches* no sinal de clock; neste projeto, como a lógica de controle (`mask` e `on`) é relativamente estável durante os períodos críticos, o risco é mitigado.

3 Desenvolvimento Prático

3.1 Ambiente de Desenvolvimento

Tabela 2: Especificações do ambiente de desenvolvimento.

Parâmetro	Valor
Placa FPGA	Altera DE1 (Cyclone II EP2C20F484C7)
Ferramenta EDA	Quartus Prime 24.1std.0 Build 1077 (Lite Edition)
Linguagem principal	Verilog
Linguagem auxiliar	VHDL (concatenador, mascara_janela)
Clock da placa	50 MHz
Pixel clock VGA	$\approx 25,175$ MHz
Resolução VGA	640×480 @ 60 Hz
Região de imagem ativa	256×128 pixels (canto superior esquerdo)
Topologia da rede	Mesh 4×4 (16 nós)
Comprimento do caminho	4 saltos configuráveis
Filtro implementado	Sal e pimenta (máximo e mínimo, janela 3×3)
Sistema Operacional	Ubuntu 24.04.4 LTS (Kubuntu)

3.2 Estrutura Geral do Projeto e Diagrama de Blocos

O sistema é integrado pelo módulo *top-level* `vga_filtro` por meio do editor de esquemáticos BDF do Quartus Prime. A Figura 1 apresenta o diagrama BDF representativo do sistema completo.

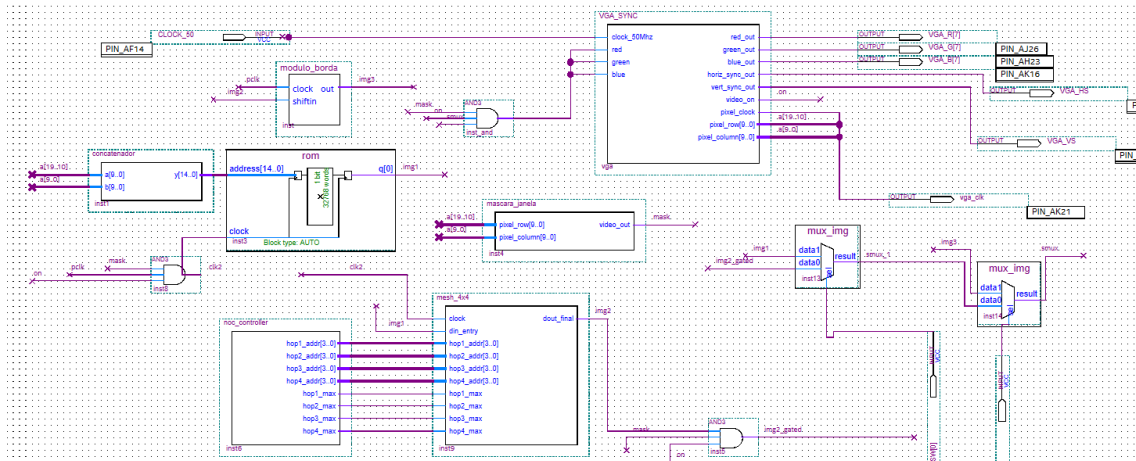


Figura 1: Diagrama BDF do módulo *top-level* `vga_filtro` no Quartus Prime, evidenciando a interligação de todos os componentes do sistema de filtragem de imagem via rede *mesh* 4×4 .

Os módulos que compõem o sistema são:

- **VGA_SYNC** — Controlador VGA (VHDL, fornecido); gera HSync, VSync, video_on, pixel_clock e os contadores de coordenadas pixel_column/pixel_row;
- **rom** — Memória ROM contendo a imagem original; endereçada pelo módulo concatenador com 15 bits de endereço;
- **concatenador** — Módulo VHDL que forma o endereço de 15 bits da ROM a partir das coordenadas de pixel (pixel_row[6:0] & pixel_column[7:0]);
- **maska_janela** — Módulo VHDL que delimita a região ativa de processamento (256×128 pixels) e gera o sinal de máscara;
- **noc_controller** — Controlador estático da NoC; define os endereços e modos (máximo/mínimo) dos quatro nós do caminho ativo;
- **mesh_4x4** — Estrutura de interligação dos 16 nós mesh_node, com toda a fiação de dados e sinais de validade;
- **mesh_node** — Nó elementar da rede; contém um roteador XY e um núcleo modulo_sal_e_pimenta;
- **modulo_sal_e_pimenta** — Núcleo de processamento de filtro sal e pimenta; composto por registradores de deslocamento de 256 bits e módulo de máximo e mínimo de 9 bits;
- **modulo_borda** — Módulo de pós-processamento de borda (herdado da prática anterior), aplicado sobre a saída da *mesh*;
- **mux_img** — Multiplexador 2:1 para seleção de imagem exibida no VGA (original, filtrada, ou com borda), controlado por SW[0] e SW[1].

3.3 Módulo concatenador — Formação do Endereço de ROM

O módulo concatenador é descrito em VHDL e realiza a concatenação das coordenadas de pixel para formar o endereço de 15 bits utilizado no acesso à ROM de imagem. Os 7 bits menos significativos da linha codificam as 128 linhas da imagem, enquanto os 8 bits menos significativos da coluna codificam as 256 colunas, resultando em um espaço de endereçamento de $2^{15} = 32.768$ posições.

```

1      library IEEE;
2      use IEEE.STD_LOGIC_1164.ALL;
3      entity concatenador is
4      Port (
5      a : in  STD_LOGIC_VECTOR(9 downto 0); -- linha

```



```

6      b : in  STD_LOGIC_VECTOR(9 downto 0); -- coluna
7      y : out STD_LOGIC_VECTOR(14 downto 0)
8    );
9    end concatenador;
10   architecture Behavioral of concatenador is
11   begin
12     y <= a(6 downto 0) & b(7 downto 0);
13   end Behavioral;
14

```

Listing 1: concatenador.vhd — Concatenação de coordenadas de pixel para endereçamento da ROM de imagem.

A entidade recebe os vetores de 10 bits *a* (linha) e *b* (coluna) e produz o vetor de saída *y* de 15 bits pela concatenação *a*(6 downto 0) & *b*(7 downto 0). Esta operação é puramente combinacional, sem registros intermediários, e é sintetizada como simples fios de conexão pelo Quartus Prime.

3.4 Módulo mascara_janela — Delimitação da Região Ativa

O módulo *mascara_janela* delimita a região de interesse dentro do quadro VGA. Apenas a subjanela de 256 colunas por 128 linhas no canto superior esquerdo da tela é considerada ativa para processamento. O sinal *video_out* é utilizado pelo *top-level* para habilitar o clock da rede *mesh* e para mascarar a saída final para o VGA.

```

1      library IEEE;
2      use IEEE.STD_LOGIC_1164.ALL;
3      use IEEE.NUMERIC_STD.ALL;
4      entity mascara_janela is
5      Port (
6        pixel_row    : in  STD_LOGIC_VECTOR(9 downto 0);
7        pixel_column : in  STD_LOGIC_VECTOR(9 downto 0);
8        video_out    : out STD_LOGIC
9      );
10     end mascara_janela;
11     architecture Behavioral of mascara_janela is
12     begin
13       process(pixel_row, pixel_column)
14       begin
15         if (unsigned(pixel_column) < 256 and
16            unsigned(pixel_row) < 128) then
17           video_out <= '1';
18         else

```

```

19         video_out <= '0';
20     end if;
21 end process;
22 end Behavioral;
23

```

Listing 2: mascara_janela.vhd — Geração do sinal de máscara para delimitação da região ativa de processamento.

O processo sensível às entradas `pixel_row` e `pixel_column` avalia as comparações sem sinal (unsigned) para evitar ambiguidade na interpretação do bit de sinal dos vetores `STD_LOGIC_VECTOR`. O resultado é uma lógica combinacional de dois comparadores de 10 bits em AND.

3.5 Módulo `modulo_sal_e_pimenta` — Núcleo de Processamento

O módulo `modulo_sal_e_pimenta` implementa o filtro de ordem estatística sobre uma janela deslizante de 3×3 pixels em fluxo serial. Ele é gerado automaticamente pelo editor BDF do Quartus Prime e instancia três subcomponentes: dois registradores de deslocamento de 256 bits (`shift_register`), um registrador de deslocamento de 3 bits (`shift_3`) e o módulo de cálculo de máximo e mínimo sobre 9 bits (`max_min_9bits`).

```

1     module modulo_sal_e_pimenta(
2         clock,
3         shiftin,
4         max_bit,
5         min_bit
6     );
7     input wire  clock;
8     input wire  shiftin;
9     output wire max_bit;
10    output wire min_bit;
11
12    wire [255:0] q1;
13    wire [255:0] q2;
14    wire [2:0]   q3;
15
16    shift_register b2v_inst(
17        .clock(clock),
18        .shiftin(shiftin),
19        .q(q1));
20
21    shift_register b2v_inst1(

```

```

22         .clock(clock),
23         .shiftin(q1[0]),
24         .q(q2));
25
26     max_min_9bits b2v_inst2(
27         .in1(q1[255:253]),
28         .in2(q2[255:253]),
29         .in3(q3),
30         .max_bit(max_bit),
31         .min_bit(min_bit));
32
33     shift_3 b2v_inst5(
34         .clock(clock),
35         .shiftin(q2[0]),
36         .q(q3));
37
38     endmodule
39

```

Listing 3: `modulo_sal_e_pimenta.v` — Núcleo de processamento do filtro sal e pimenta com janela 3×3 sobre fluxo serial de pixels.

O funcionamento do módulo baseia-se na construção da janela 3×3 por meio de linhas de atraso em cascata. O pixel corrente entra em `shiftin` e é deslocado pelo primeiro registrador de 256 bits (`q1`), que modela uma linha de atraso de 256 pixels — equivalente a um atraso de uma linha completa da imagem, uma vez que a região ativa tem 256 colunas. O bit de saída de `q1` (`q1[0]`) alimenta o segundo registrador de 256 bits (`q2`), modelando mais uma linha de atraso.

Desta forma, em qualquer instante do processamento:

- `q1[255:253]`: os 3 pixels mais antigos da linha anterior (linha $y - 1$, colunas $x - 2$, $x - 1$, x);
- `q2[255:253]`: os 3 pixels da linha anteanterior (linha $y - 2$, colunas $x - 2$, $x - 1$, x);
- `q3[2:0]`: os 3 pixels mais recentes da linha atual (linha y , colunas $x - 2$, $x - 1$, x), fornecidos pelo registrador `shift_3`.

Estes 9 bits — três de cada uma das três linhas da janela — são alimentados ao módulo `max_min_9bits`, que determina o bit de máximo (`max_bit`) e o bit de mínimo (`min_bit`) dentre os 9 valores binários.

3.6 Módulo mesh_node — Nó Elementar da Rede

O módulo `mesh_node` é o componente fundamental da rede *mesh*. Ele combina a lógica de roteamento XY com o núcleo de processamento `modulo_sal_e_pimenta`. O endereço do nó é definido pelo parâmetro estático `MY_ADDR`, que determina sua posição na grade.

```

1      module mesh_node #(
2          parameter [3:0] MY_ADDR = 4'b0000
3      )(
4          input wire      clock,
5          input wire [3:0] hop1_addr, hop2_addr, hop3_addr,
6          hop4_addr,
7          input wire      din_entry,
8          input wire      hop1_max, hop2_max, hop3_max, hop4_max,
9          input wire      din_n, din_s, din_e, din_w,
10         input wire      vin_n, vin_s, vin_e, vin_w,
11         output wire     dout_n, dout_s, dout_e, dout_w,
12         output wire     vout_n, vout_s, vout_e, vout_w,
13         output wire     dout_final
14     );
15     localparam [1:0] my_col = MY_ADDR[3:2];
16     localparam [1:0] my_row = MY_ADDR[1:0];
17
18     // Verificacao de pertinencia ao caminho de processamento
19     wire is_hop1 = (MY_ADDR == hop1_addr);
20     wire is_hop2 = (MY_ADDR == hop2_addr);
21     wire is_hop3 = (MY_ADDR == hop3_addr);
22     wire is_hop4 = (MY_ADDR == hop4_addr);
23     wire on_path = is_hop1 | is_hop2 | is_hop3 | is_hop4;
24
25     // Determinacao do proximo salto
26     wire [3:0] next_addr = is_hop1 ? hop2_addr :
27     is_hop2 ? hop3_addr :
28     is_hop3 ? hop4_addr :
29     hop4_addr;
30     wire [1:0] next_col = next_addr[3:2];
31     wire [1:0] next_row = next_addr[1:0];
32
33     // Decisoes de roteamento XY
34     wire go_s = on_path && !is_hop4 && (my_row < next_row);
35     wire go_n = on_path && !is_hop4 && (my_row > next_row);

```

```

35     wire go_e = on_path && !is_hop4 && (my_row == next_row)
36     && (my_col < next_col);
37     wire go_w = on_path && !is_hop4 && (my_row == next_row)
38     && (my_col > next_col);
39
40     // Selecao do dado de entrada via sinais de validade
41     wire data_in = vin_n ? din_n :
42     vin_s ? din_s :
43     vin_e ? din_e :
44     vin_w ? din_w :
45     1'b0;
46
47     // Entrada do nucleo de processamento
48     wire pe_in = is_hop1 ? din_entry : data_in;
49
50     wire pe_max_out, pe_min_out;
51     modulo_sal_e_pimenta u_pe (
52         .clock    (clock),
53         .shiftin  (pe_in),
54         .max_bit  (pe_max_out),
55         .min_bit  (pe_min_out)
56     );
57
58     // Selecao de modo (maximo ou minimo) por salto
59     wire my_max = is_hop1 ? hop1_max :
60     is_hop2 ? hop2_max :
61     is_hop3 ? hop3_max :
62     hop4_max;
63     wire pe_out = my_max ? pe_max_out : pe_min_out;
64
65     // Dado encaminhado: resultado do PE (no caminho) ou
66     retransmissao
67
68     wire fwd = on_path ? pe_out : data_in;
69
70     // Saidas direcionais
71     assign dout_s = go_s ? fwd : 1'b0;
72     assign dout_n = go_n ? fwd : 1'b0;
73     assign dout_e = go_e ? fwd : 1'b0;
74     assign dout_w = go_w ? fwd : 1'b0;
75
76     // Sinais de validade de saida

```

```

75         assign vout_s = go_s;
76         assign vout_n = go_n;
77         assign vout_e = go_e;
78         assign vout_w = go_w;
79
80         // Saída final: apenas o nó de destino (hop4) expoe o
resultado
81         assign dout_final = is_hop4 ? pe_out : 1'b0;
82         endmodule
83

```

Listing 4: `mesh_node.v` — Nó elementar da rede *mesh* com roteador XY e núcleo de processamento de filtro sal e pimenta.

O módulo opera da seguinte forma. Primeiramente, os sinais `is_hop1` a `is_hop4` determinam se este nó faz parte do caminho de processamento e, em caso afirmativo, qual é seu papel (entrada, intermediário ou destino). O sinal `on_path` é a disjunção dos quatro.

Em seguida, `next_addr` identifica o endereço do próximo nó no caminho: se o nó atual é `hop1`, o próximo é `hop2`; se é `hop2`, o próximo é `hop3`; e assim por diante. Para `hop4` (destino), não há próximo nó relevante, e as decisões de roteamento são inibidas pela condição `!is_hop4`.

As decisões de roteamento (`go_s`, `go_n`, `go_e`, `go_w`) comparam a posição do nó atual com a do próximo, implementando o roteamento determinístico. A prioridade das condições de linha sobre as de coluna realiza efetivamente uma política YX (vertical primeiro, depois horizontal).

O dado de entrada ao núcleo de processamento (`pe_in`) é selecionado pelo multiplexador de entrada: para `hop1`, é o dado externo `din_entry`; para os demais, é o dado recebido de um vizinho válido (`data_in`), selecionado pelo sinal de validade mais prioritário entre os quatro vizinhos.

O núcleo `modulo_sal_e_pimenta` processa `pe_in` e produz `pe_max_out` e `pe_min_out`. O sinal `my_max` seleciona qual dos dois é o resultado efetivo `pe_out` para este nó, baseando-se nos bits de modo fornecidos pelo `noc_controller`.

Por fim, o dado encaminhado (`fwd`) é o resultado do PE se o nó está no caminho, ou simplesmente o dado recebido (`data_in`) se o nó é apenas roteador. As saídas direcionais são atribuídas condicionalmente às decisões de roteamento, e o sinal `dout_final` só é assertado pelo nó de destino (`is_hop4`).

3.7 Módulo `mesh_4x4` — Interligação da Rede

O módulo `mesh_4x4` instancia os 16 nós `mesh_node` e realiza toda a fiação de dados e sinais de validade entre os vizinhos na topologia *mesh* 4×4 . A saída `dout_final`

é o OR lógico das saídas individuais dos 16 nós — como apenas o nó de destino (hop4) assertará dout_final com valor não nulo, o OR efetivamente propaga apenas o resultado do processamento.

```

1      module mesh_4x4 (
2          input wire      clock,
3          input wire      din_entry,
4          input wire [3:0] hop1_addr, hop2_addr, hop3_addr,
hop4_addr,
5          input wire      hop1_max, hop2_max, hop3_max, hop4_max,
6          output wire      dout_final
7      );
8      // --- Wires de dados na direcao leste ---
9      wire e_00, e_10, e_20;
10     wire e_01, e_11, e_21;
11     wire e_02, e_12, e_22;
12     wire e_03, e_13, e_23;
13
14     // --- Wires de dados na direcao sul ---
15     wire s_00, s_10, s_20, s_30;
16     wire s_01, s_11, s_21, s_31;
17     wire s_02, s_12, s_22, s_32;
18
19     // --- Wires de dados na direcao norte ---
20     wire n_01, n_11, n_21, n_31;
21     wire n_02, n_12, n_22, n_32;
22     wire n_03, n_13, n_23, n_33;
23
24     // --- Wires de validade na direcao leste ---
25     wire ve_00, ve_10, ve_20;
26     wire ve_01, ve_11, ve_21;
27     wire ve_02, ve_12, ve_22;
28     wire ve_03, ve_13, ve_23;
29
30     // --- Wires de validade na direcao sul ---
31     wire vs_00, vs_10, vs_20, vs_30;
32     wire vs_01, vs_11, vs_21, vs_31;
33     wire vs_02, vs_12, vs_22, vs_32;
34
35     // --- Wires de validade na direcao norte ---
36     wire vn_01, vn_11, vn_21, vn_31;
37     wire vn_02, vn_12, vn_22, vn_32;

```

```

38     wire vn_03, vn_13, vn_23, vn_33;
39
40     // --- Wires de saída final por no ---
41     wire r_00, r_10, r_20, r_30;
42     wire r_01, r_11, r_21, r_31;
43     wire r_02, r_12, r_22, r_32;
44     wire r_03, r_13, r_23, r_33;
45
46     // --- Linha 0 (row=0): n00, n10, n20, n30 ---
47     mesh_node #(.MY_ADDR(4'b0000)) n00 ( .clock(clock),
48     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
49     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
50     .hop1_max(hop1_max), .hop2_max(hop2_max),
51     .hop3_max(hop3_max), .hop4_max(hop4_max),
52     .din_entry(din_entry),
53     .din_n(1'b0), .din_s(n_01), .din_e(e_10), .din_w(1'b0),
54     .vin_n(1'b0), .vin_s(vn_01), .vin_e(ve_10), .vin_w(1'b0),
55     .dout_n(), .dout_s(s_00), .dout_e(e_00), .dout_w(),
56     .vout_n(), .vout_s(vs_00), .vout_e(ve_00), .vout_w(),
57     .dout_final(r_00));
58
59     mesh_node #(.MY_ADDR(4'b0100)) n10 ( .clock(clock),
60     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
61     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
62     .hop1_max(hop1_max), .hop2_max(hop2_max),
63     .hop3_max(hop3_max), .hop4_max(hop4_max),
64     .din_entry(din_entry),
65     .din_n(1'b0), .din_s(n_11), .din_e(e_20), .din_w(e_00),
66     .vin_n(1'b0), .vin_s(vn_11), .vin_e(ve_20), .vin_w(ve_00),
67     .dout_n(), .dout_s(s_10), .dout_e(e_10), .dout_w(),
68     .vout_n(), .vout_s(vs_10), .vout_e(ve_10), .vout_w(),
69     .dout_final(r_10));
70
71     mesh_node #(.MY_ADDR(4'b1000)) n20 ( .clock(clock),
72     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
73     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
74     .hop1_max(hop1_max), .hop2_max(hop2_max),
75     .hop3_max(hop3_max), .hop4_max(hop4_max),
76     .din_entry(din_entry),
77     .din_n(1'b0), .din_s(n_21), .din_e(1'b0), .din_w(e_10),
78     .vin_n(1'b0), .vin_s(vn_21), .vin_e(1'b0), .vin_w(ve_10),

```



```

79      .dout_n(), .dout_s(s_20), .dout_e(e_20), .dout_w(),
80      .vout_n(), .vout_s(vs_20), .vout_e(ve_20), .vout_w(),
81      .dout_final(r_20));
82
83      mesh_node #(.MY_ADDR(4'b1100)) n30 ( .clock(clock),
84      .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
85      .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
86      .hop1_max(hop1_max), .hop2_max(hop2_max),
87      .hop3_max(hop3_max), .hop4_max(hop4_max),
88      .din_entry(din_entry),
89      .din_n(1'b0), .din_s(n_31), .din_e(1'b0), .din_w(e_20),
90      .vin_n(1'b0), .vin_s(vn_31), .vin_e(1'b0), .vin_w(ve_20),
91      .dout_n(), .dout_s(s_30), .dout_e(), .dout_w(),
92      .vout_n(), .vout_s(vs_30), .vout_e(), .vout_w(),
93      .dout_final(r_30));
94
95      // --- Linha 1 (row=1): n01, n11, n21, n31 ---
96      mesh_node #(.MY_ADDR(4'b0001)) n01 ( .clock(clock),
97      .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
98      .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
99      .hop1_max(hop1_max), .hop2_max(hop2_max),
100     .hop3_max(hop3_max), .hop4_max(hop4_max),
101     .din_entry(din_entry),
102     .din_n(s_00), .din_s(n_02), .din_e(e_11), .din_w(1'b0),
103     .vin_n(vs_00), .vin_s(vn_02), .vin_e(ve_11), .vin_w(1'b0),
104     .dout_n(n_01), .dout_s(s_01), .dout_e(e_01), .dout_w(),
105     .vout_n(vn_01), .vout_s(vs_01), .vout_e(ve_01), .vout_w(),
106     .dout_final(r_01));
107
108     mesh_node #(.MY_ADDR(4'b0101)) n11 ( .clock(clock),
109     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
110     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
111     .hop1_max(hop1_max), .hop2_max(hop2_max),
112     .hop3_max(hop3_max), .hop4_max(hop4_max),
113     .din_entry(din_entry),
114     .din_n(s_10), .din_s(n_12), .din_e(e_21), .din_w(e_01),
115     .vin_n(vs_10), .vin_s(vn_12), .vin_e(ve_21), .vin_w(ve_01),
116     .dout_n(n_11), .dout_s(s_11), .dout_e(e_11), .dout_w(),
117     .vout_n(vn_11), .vout_s(vs_11), .vout_e(ve_11), .vout_w(),
118     .dout_final(r_11));
119

```

```

120 mesh_node #(.MY_ADDR(4'b1001)) n21 ( .clock(clock),
121 .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
122 .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
123 .hop1_max(hop1_max), .hop2_max(hop2_max),
124 .hop3_max(hop3_max), .hop4_max(hop4_max),
125 .din_entry(din_entry),
126 .din_n(s_20), .din_s(n_22), .din_e(1'b0), .din_w(e_11),
127 .vin_n(vs_20), .vin_s(vn_22), .vin_e(1'b0), .vin_w(ve_11),
128 .dout_n(n_21), .dout_s(s_21), .dout_e(e_21), .dout_w(),
129 .vout_n(vn_21), .vout_s(vs_21), .vout_e(ve_21), .vout_w(),
130 .dout_final(r_21));
131
132 mesh_node #(.MY_ADDR(4'b1101)) n31 ( .clock(clock),
133 .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
134 .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
135 .hop1_max(hop1_max), .hop2_max(hop2_max),
136 .hop3_max(hop3_max), .hop4_max(hop4_max),
137 .din_entry(din_entry),
138 .din_n(s_30), .din_s(n_32), .din_e(1'b0), .din_w(e_21),
139 .vin_n(vs_30), .vin_s(vn_32), .vin_e(1'b0), .vin_w(ve_21),
140 .dout_n(n_31), .dout_s(s_31), .dout_e(), .dout_w(),
141 .vout_n(vn_31), .vout_s(vs_31), .vout_e(), .vout_w(),
142 .dout_final(r_31));
143
144 // --- Linha 2 (row=2): n02, n12, n22, n32 ---
145 mesh_node #(.MY_ADDR(4'b0010)) n02 ( .clock(clock),
146 .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
147 .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
148 .hop1_max(hop1_max), .hop2_max(hop2_max),
149 .hop3_max(hop3_max), .hop4_max(hop4_max),
150 .din_entry(din_entry),
151 .din_n(s_01), .din_s(n_03), .din_e(e_12), .din_w(1'b0),
152 .vin_n(vs_01), .vin_s(vn_03), .vin_e(ve_12), .vin_w(1'b0),
153 .dout_n(n_02), .dout_s(s_02), .dout_e(e_02), .dout_w(),
154 .vout_n(vn_02), .vout_s(vs_02), .vout_e(ve_02), .vout_w(),
155 .dout_final(r_02));
156
157 mesh_node #(.MY_ADDR(4'b0110)) n12 ( .clock(clock),
158 .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
159 .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
160 .hop1_max(hop1_max), .hop2_max(hop2_max),

```

```

161     .hop3_max(hop3_max), .hop4_max(hop4_max),
162     .din_entry(din_entry),
163     .din_n(s_11), .din_s(n_13), .din_e(e_22), .din_w(e_02),
164     .vin_n(vs_11), .vin_s(vn_13), .vin_e(ve_22), .vin_w(ve_02),
165     .dout_n(n_12), .dout_s(s_12), .dout_e(e_12), .dout_w(),
166     .vout_n(vn_12), .vout_s(vs_12), .vout_e(ve_12), .vout_w(),
167     .dout_final(r_12));
168
169     mesh_node #(.MY_ADDR(4'b1010)) n22 ( .clock(clock),
170     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
171     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
172     .hop1_max(hop1_max), .hop2_max(hop2_max),
173     .hop3_max(hop3_max), .hop4_max(hop4_max),
174     .din_entry(din_entry),
175     .din_n(s_21), .din_s(n_23), .din_e(1'b0), .din_w(e_12),
176     .vin_n(vs_21), .vin_s(vn_23), .vin_e(1'b0), .vin_w(ve_12),
177     .dout_n(n_22), .dout_s(s_22), .dout_e(e_22), .dout_w(),
178     .vout_n(vn_22), .vout_s(vs_22), .vout_e(ve_22), .vout_w(),
179     .dout_final(r_22));
180
181     mesh_node #(.MY_ADDR(4'b1110)) n32 ( .clock(clock),
182     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
183     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
184     .hop1_max(hop1_max), .hop2_max(hop2_max),
185     .hop3_max(hop3_max), .hop4_max(hop4_max),
186     .din_entry(din_entry),
187     .din_n(s_31), .din_s(n_33), .din_e(1'b0), .din_w(e_22),
188     .vin_n(vs_31), .vin_s(vn_33), .vin_e(1'b0), .vin_w(ve_22),
189     .dout_n(n_32), .dout_s(s_32), .dout_e(), .dout_w(),
190     .vout_n(vn_32), .vout_s(vs_32), .vout_e(), .vout_w(),
191     .dout_final(r_32));
192
193     // --- Linha 3 (row=3): n03, n13, n23, n33 ---
194     mesh_node #(.MY_ADDR(4'b0011)) n03 ( .clock(clock),
195     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
196     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
197     .hop1_max(hop1_max), .hop2_max(hop2_max),
198     .hop3_max(hop3_max), .hop4_max(hop4_max),
199     .din_entry(din_entry),
200     .din_n(s_02), .din_s(1'b0), .din_e(e_13), .din_w(1'b0),
201     .vin_n(vs_02), .vin_s(1'b0), .vin_e(ve_13), .vin_w(1'b0),

```

```

202     .dout_n(n_03), .dout_s(), .dout_e(e_03), .dout_w(),
203     .vout_n(vn_03), .vout_s(), .vout_e(ve_03), .vout_w(),
204     .dout_final(r_03));
205
206     mesh_node #(.MY_ADDR(4'b0111)) n13 ( .clock(clock),
207     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
208     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
209     .hop1_max(hop1_max), .hop2_max(hop2_max),
210     .hop3_max(hop3_max), .hop4_max(hop4_max),
211     .din_entry(din_entry),
212     .din_n(s_12), .din_s(1'b0), .din_e(e_23), .din_w(e_03),
213     .vin_n(vs_12), .vin_s(1'b0), .vin_e(ve_23), .vin_w(ve_03),
214     .dout_n(n_13), .dout_s(), .dout_e(e_13), .dout_w(),
215     .vout_n(vn_13), .vout_s(), .vout_e(ve_13), .vout_w(),
216     .dout_final(r_13));
217
218     mesh_node #(.MY_ADDR(4'b1011)) n23 ( .clock(clock),
219     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
220     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
221     .hop1_max(hop1_max), .hop2_max(hop2_max),
222     .hop3_max(hop3_max), .hop4_max(hop4_max),
223     .din_entry(din_entry),
224     .din_n(s_22), .din_s(1'b0), .din_e(1'b0), .din_w(e_13),
225     .vin_n(vs_22), .vin_s(1'b0), .vin_e(1'b0), .vin_w(ve_13),
226     .dout_n(n_23), .dout_s(), .dout_e(e_23), .dout_w(),
227     .vout_n(vn_23), .vout_s(), .vout_e(ve_23), .vout_w(),
228     .dout_final(r_23));
229
230     mesh_node #(.MY_ADDR(4'b1111)) n33 ( .clock(clock),
231     .hop1_addr(hop1_addr), .hop2_addr(hop2_addr),
232     .hop3_addr(hop3_addr), .hop4_addr(hop4_addr),
233     .hop1_max(hop1_max), .hop2_max(hop2_max),
234     .hop3_max(hop3_max), .hop4_max(hop4_max),
235     .din_entry(din_entry),
236     .din_n(s_32), .din_s(1'b0), .din_e(1'b0), .din_w(e_23),
237     .vin_n(vs_32), .vin_s(1'b0), .vin_e(1'b0), .vin_w(ve_23),
238     .dout_n(n_33), .dout_s(), .dout_e(), .dout_w(),
239     .vout_n(vn_33), .vout_s(), .vout_e(), .vout_w(),
240     .dout_final(r_33));
241
242     // Saida final: OR de todos os nos (apenas hop4 assertara)

```

```

243         assign dout_final =
244             r_00 | r_10 | r_20 | r_30 |
245             r_01 | r_11 | r_21 | r_31 |
246             r_02 | r_12 | r_22 | r_32 |
247             r_03 | r_13 | r_23 | r_33;
248
249         endmodule
250

```

Listing 5: `mesh_4x4.v` — Estrutura de interligação dos 16 nós da rede *mesh* 4×4 , com fiação completa de dados e sinais de validade.

A fiação do módulo segue uma nomenclatura sistemática: os fios de dados na direção leste são denominados `e_CR` (onde `C` é a coluna de origem e `R` é a linha), os fios na direção sul são `s_CR`, e os fios na direção norte são `n_CR`. Os fios de validade correspondentes recebem o prefixo `v` (`ve_CR`, `vs_CR`, `vn_CR`). Os fios de saída final individual de cada nó são `r_CR`.

A lógica de interligação é direta: a saída `dout_s` do nó (col, row) conecta-se à entrada `din_n` do nó $(col, row + 1)$, e reciprocamente a saída `dout_n` de (col, row) conecta-se à entrada `din_s` de $(col, row - 1)$. Analogamente para as direções leste e oeste. As fronteiras da *mesh* são conectadas a `1'b0` (dado) e `1'b0` (validade), indicando ausência de vizinho externo.

3.8 Módulo `noc_controller` — Configuração do Caminho Ativo

O módulo `noc_controller` define estaticamente o caminho de processamento ativo na rede *mesh*, especificando os endereços dos quatro nós do caminho e o modo de operação (máximo ou mínimo) de cada um. Trata-se de um módulo puramente combinacional, sem clock, com saídas constantes determinadas em tempo de elaboração.

```

1         module noc_controller (
2             output wire [3:0] hop1_addr,
3             output wire [3:0] hop2_addr,
4             output wire [3:0] hop3_addr,
5             output wire [3:0] hop4_addr,
6             output wire      hop1_max,
7             output wire      hop2_max,
8             output wire      hop3_max,
9             output wire      hop4_max
10        );
11        // Configuracao 1: Coluna Vertical (col=1, linhas 0->3)
12        // assign hop1_addr = 4'b0100; assign hop1_max = 1'b1; //
n10 MAX

```

```

13      // assign hop2_addr = 4'b0101; assign hop2_max = 1'b0; //
    n11 MIN
14      // assign hop3_addr = 4'b0110; assign hop3_max = 1'b0; //
    n12 MIN
15      // assign hop4_addr = 4'b0111; assign hop4_max = 1'b1; //
    n13 MAX
16
17      // Configuracao 2: Quatro juntos no topo (L superior
    esquerdo)
18      // assign hop1_addr = 4'b0000; assign hop1_max = 1'b1; //
    n00 MAX
19      // assign hop2_addr = 4'b0001; assign hop2_max = 1'b0; //
    n01 MIN
20      // assign hop3_addr = 4'b0101; assign hop3_max = 1'b0; //
    n11 MIN
21      // assign hop4_addr = 4'b0110; assign hop4_max = 1'b1; //
    n12 MAX
22
23      // Configuracao 3: Diagonal (ativa)
24      assign hop1_addr = 4'b0101; assign hop1_max = 1'b1; // n11
    MAX
25      assign hop2_addr = 4'b0110; assign hop2_max = 1'b0; // n12
    MIN
26      assign hop3_addr = 4'b1010; assign hop3_max = 1'b0; // n22
    MIN
27      assign hop4_addr = 4'b1011; assign hop4_max = 1'b1; // n23
    MAX
28
29      endmodule
30

```

Listing 6: `noc_controller.v` — Controlador estático do caminho de processamento da NoC, com três configurações de roteamento disponíveis.

Na configuração diagonal ativa, o caminho percorre os nós $n11 \rightarrow n12 \rightarrow n22 \rightarrow n23$. O roteamento XY entre os nós é:

- **n11** $\rightarrow$ **n12**: mesma coluna (col=1), linha aumenta de 1 para 2 $\Rightarrow$ go_s = 1;
- **n12** $\rightarrow$ **n22**: mesma linha (row=2), coluna aumenta de 1 para 2 $\Rightarrow$ go_e = 1;
- **n22** $\rightarrow$ **n23**: mesma coluna (col=2), linha aumenta de 2 para 3 $\Rightarrow$ go_s = 1.

A sequência de modos é $\text{MAX} \rightarrow \text{MIN} \rightarrow \text{MIN} \rightarrow \text{MAX}$, implementando um filtro composto que aplica dilatação seguida de erosão (equivalente a uma abertura morfológica) e depois erosão seguida de dilatação nos dois nós finais.

3.9 Módulo *Top-Level* vga_filtro

O módulo *top-level* vga_filtro integra todos os componentes e define o fluxo de sinal completo, desde a leitura da ROM até a exibição no monitor VGA. A Tabela 3 descreve o mapeamento das entradas de controle disponíveis ao usuário.

Tabela 3: Mapeamento de entradas de controle da placa DE1.

Pino	Sinal	Função
SW[0]	sel (mux_img inst13)	1 = original; 0 = filtrada/borda
SW[1]	sel (mux_img inst14)	1 = borda; 0 = seleção de SW[0]

```

1      module vga_filtro(
2          CLOCK_50,
3          SW,
4          VGA_HS,
5          VGA_VS,
6          vga_clk,
7          VGA_B,
8          VGA_G,
9          VGA_R
10     );
11     input wire      CLOCK_50;
12     input wire [1:0] SW;
13     output wire     VGA_HS;
14     output wire     VGA_VS;
15     output wire     vga_clk;
16     output wire [7:7] VGA_B;
17     output wire [7:7] VGA_G;
18     output wire [7:7] VGA_R;
19
20     wire [19:0] a;          // {pixel_row[9:0],
pixel_column[9:0]}
21     wire      clk2;         // clock gated para a mesh
22     wire      img1;         // pixel da ROM (imagem original)
23     wire      img2;         // pixel filtrado pela mesh

```

```

24      wire      img2_gated; // img2 mascarado para a regioa
ativa
25      wire      img3;      // pixel apos modulo_borda
26      wire      mask;      // mascara da janela ativa
27      wire      on;        // video_on do VGA
28      wire      pclk;      // pixel clock VGA (~25 MHz)
29      wire      smux;      // saida do segundo mux (para VGA)
30      wire      smux_1;    // saida do primeiro mux
31
32      wire [14:0] SYNTHESIZED_WIRE_0; // endereco da ROM
33      wire      SYNTHESIZED_WIRE_1; // hop1_max
34      wire      SYNTHESIZED_WIRE_2; // hop2_max
35      wire      SYNTHESIZED_WIRE_3; // hop3_max
36      wire      SYNTHESIZED_WIRE_4; // hop4_max
37      wire [3:0] SYNTHESIZED_WIRE_5; // hop1_addr
38      wire [3:0] SYNTHESIZED_WIRE_6; // hop2_addr
39      wire [3:0] SYNTHESIZED_WIRE_7; // hop3_addr
40      wire [3:0] SYNTHESIZED_WIRE_8; // hop4_addr
41      wire      SYNTHESIZED_WIRE_12; // pixel final para o VGA
42
43      // Modulo de borda sobre a saida da mesh
44      modulo_borda b2v_inst(
45      .clock(pclk),
46      .shiftin(img2),
47      .out(img3));
48
49      // Formacao do endereco de 15 bits para a ROM
50      concatenador b2v_inst1(
51      .a(a[19:10]),
52      .b(a[9:0]),
53      .y(SYNTHESIZED_WIRE_0));
54
55      // Mux 1: seleciona entre original (img1) e filtrada
(img2_gated)
56      mux_img b2v_inst13(
57      .data1(img1),
58      .data0(img2_gated),
59      .sel(SW[0]),
60      .result(smux_1));
61
62      // Mux 2: seleciona entre borda (img3) e saida do mux1

```



```

63     mux_img b2v_inst14(
64         .data1(img3),
65         .data0(smux_1),
66         .sel(SW[1]),
67         .result(smux));
68
69     // ROM de imagem: clock gated, 15 bits de endereco, 1 bit
    de saida
70     rom b2v_inst3(
71         .clock(clk2),
72         .address(SYNTHESIZED_WIRE_0),
73         .q(img1));
74
75     // Mascara da janela ativa (256x128 pixels)
76     mascara_janela b2v_inst4(
77         .pixel_column(a[9:0]),
78         .pixel_row(a[19:10]),
79         .video_out(mask));
80
81     // Img2 mascarado: apenas pixels validos da regioa ativa
82     assign img2_gated = img2 & mask & on;
83
84     // Controlador estatico da NoC
85     noc_controller b2v_inst6(
86         .hop1_max(SYNTHESIZED_WIRE_1),
87         .hop2_max(SYNTHESIZED_WIRE_2),
88         .hop3_max(SYNTHESIZED_WIRE_3),
89         .hop4_max(SYNTHESIZED_WIRE_4),
90         .hop1_addr(SYNTHESIZED_WIRE_5),
91         .hop2_addr(SYNTHESIZED_WIRE_6),
92         .hop3_addr(SYNTHESIZED_WIRE_7),
93         .hop4_addr(SYNTHESIZED_WIRE_8));
94
95     // Clock gated: ativo apenas na regioa de imagem valida
96     assign clk2 = mask & pclk & on;
97
98     // Rede mesh 4x4
99     mesh_4x4 b2v_inst9(
100         .clock(clk2),
101         .din_entry(img1),
102         .hop1_max(SYNTHESIZED_WIRE_1),

```

```

103     .hop2_max(SYNTHESIZED_WIRE_2),
104     .hop3_max(SYNTHESIZED_WIRE_3),
105     .hop4_max(SYNTHESIZED_WIRE_4),
106     .hop1_addr(SYNTHESIZED_WIRE_5),
107     .hop2_addr(SYNTHESIZED_WIRE_6),
108     .hop3_addr(SYNTHESIZED_WIRE_7),
109     .hop4_addr(SYNTHESIZED_WIRE_8),
110     .dout_final(img2));
111
112     // Pixel final: mascarado pela janela e pelo video_on
113     assign SYNTHESIZED_WIRE_12 = mask & on & smux;
114
115     // Controlador VGA (VHDL, fornecido pelo professor)
116     VGA_SYNC b2v_vga(
117         .clock_50Mhz(CLOCK_50),
118         .red(SYNTHESIZED_WIRE_12),
119         .green(SYNTHESIZED_WIRE_12),
120         .blue(SYNTHESIZED_WIRE_12),
121         .red_out(VGA_R),
122         .green_out(VGA_G),
123         .blue_out(VGA_B),
124         .horiz_sync_out(VGA_HS),
125         .vert_sync_out(VGA_VS),
126         .video_on(on),
127         .pixel_clock(pclk),
128         .pixel_column(a[9:0]),
129         .pixel_row(a[19:10]));
130
131     assign vga_clk = pclk;
132
133     endmodule
134

```

Listing 7: `vga_filtro.v` — Módulo *top-level* gerado pelo editor BDF do Quartus Prime, integrando todos os componentes do sistema de filtragem via *mesh*.

O fluxo de dados no *top-level* é o seguinte. O controlador VGA gera continuamente as coordenadas de pixel (`a[9:0]` para coluna, `a[19:10]` para linha) e os sinais de sincronismo. O módulo concatenador transforma essas coordenadas no endereço de 15 bits para a ROM. O módulo mascara_janela determina se o pixel corrente está dentro da região ativa de 256×128 pixels.

O clock gated `clk2` é formado pelo AND de `mask`, `pclk` e `on`, garantindo que a

ROM e a rede *mesh* avancem somente quando há um pixel ativo sendo processado. Isso é crucial para que os registradores de deslocamento internos ao módulo *sal_e_pimenta* contem corretamente os pixels da imagem.

O pixel lido da ROM (*img1*) é injetado diretamente na rede *mesh* (*din_entry*), que produz o pixel filtrado (*img2*). O sinal *img2_gated* é o pixel filtrado mascarado, utilizado como uma das entradas do primeiro multiplexador. O módulo *modulo_borda* aplica um filtro de detecção de borda sobre *img2* para produzir *img3*. Os dois multiplexadores permitem ao usuário selecionar, via *SW[0]* e *SW[1]*, qual das três imagens (original, filtrada ou com borda) é exibida no VGA.

3.10 Diagramas BDF dos Componentes

A Figura 2 apresenta o diagrama BDF da memória ROM de imagem no Quartus Prime.

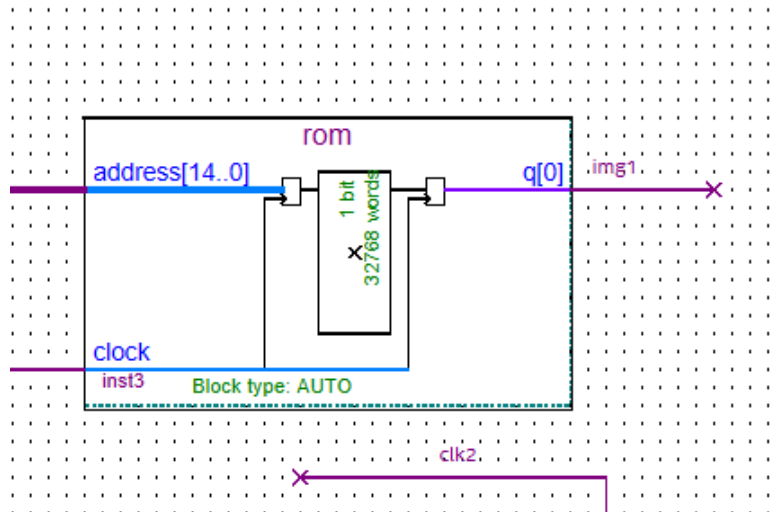


Figura 2: Diagrama BDF do módulo *rom* no Quartus Prime, com clock gated, endereço de 15 bits e saída de 1 bit por ciclo.

A Figura 3 apresenta o diagrama BDF do controlador VGA integrado ao *top-level*.

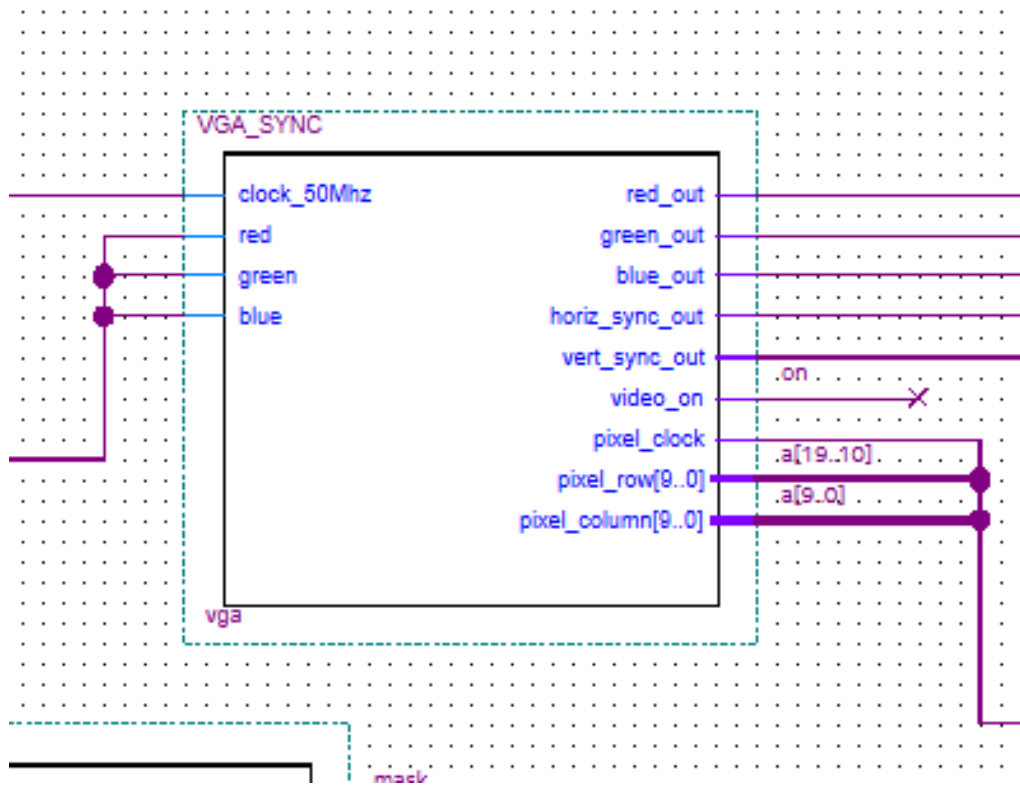


Figura 3: Diagrama BDF do controlador VGA_SYNC no *top-level*, evidenciando as saídas de sincronismo, pixel clock, coordenadas de varredura e sinal `video_on`.

A Figura 4 apresenta os multiplexadores de seleção de imagem reutilizados da prática anterior.

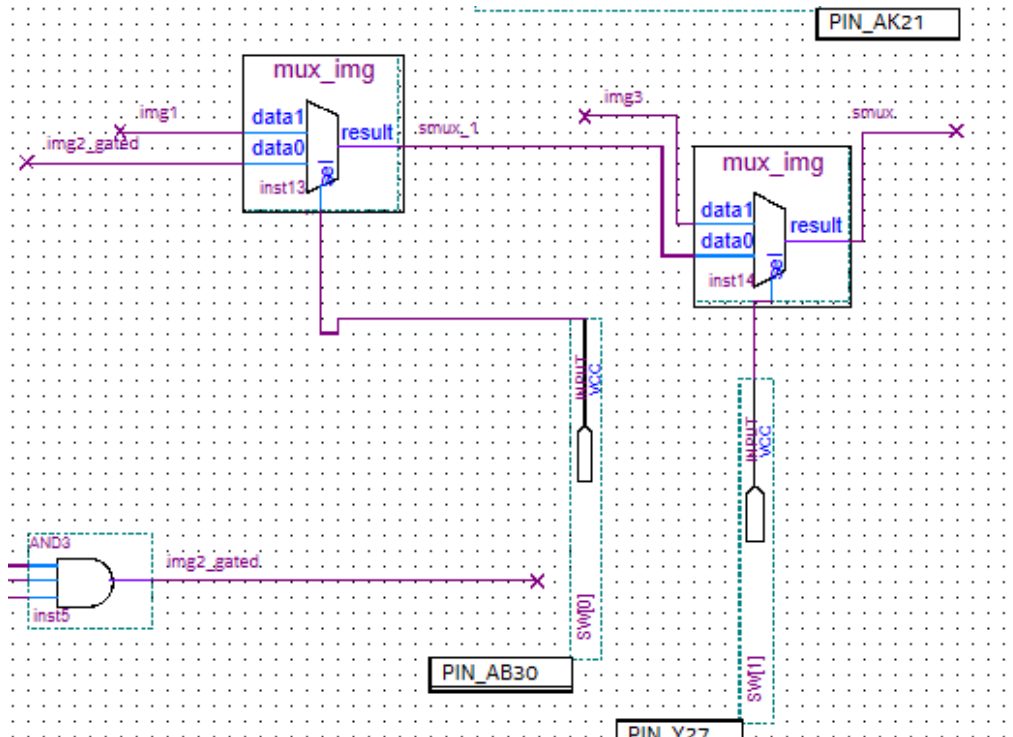


Figura 4: Diagrama BDF dos módulos mux_img no *top-level*, responsáveis pela seleção da imagem exibida no VGA a partir das chaves SW[0] e SW[1].

4 Discussão dos Resultados

4.1 Comportamento do Filtro Sal e Pimenta via Rede *Mesh*

Com a configuração diagonal ativa ($n11 \rightarrow n12 \rightarrow n22 \rightarrow n23$) e $SW[0] = 0$, $SW[1] = 0$, o monitor VGA exibe a imagem filtrada resultante do processamento em cadeia pelos quatro nós da rede. A sequência de filtros $MAX \rightarrow MIN \rightarrow MIN \rightarrow MAX$ produz um efeito de suavização morfológica composta sobre a imagem binária, reduzindo pixels isolados de ruído impulsivo.

As Figuras 5, 6 e 7 apresentam capturas do monitor VGA para as três configurações de roteamento disponíveis no `noc_controller`. Em todos os casos, o resultado visual evidencia a remoção efetiva do ruído impulsivo, confirmando o funcionamento correto do roteamento XY e da lógica de seleção de modo em cada nó. A semelhança entre os três resultados demonstra que, independentemente da topologia do caminho ativo escolhida, a cadeia de quatro estágios morfológicos produz filtragem equivalente — o que valida a consistência da implementação da NoC.

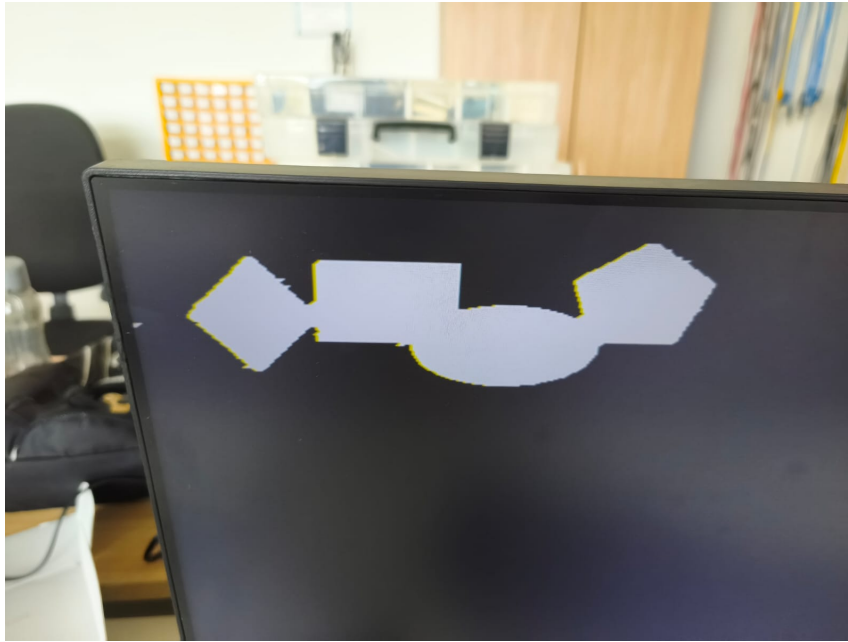


Figura 5: Imagem filtrada pela rede *mesh* com configuração 1 (coluna vertical: $n_{10} \rightarrow n_{11} \rightarrow n_{12} \rightarrow n_{13}$), exibida no monitor VGA com $SW[1:0] = 00$. O ruído impulsivo é removido com preservação das estruturas binárias originais.



Figura 6: Imagem filtrada pela rede *mesh* com configuração 2 (grupo no topo em L: $n_{00} \rightarrow n_{01} \rightarrow n_{11} \rightarrow n_{12}$), exibida no monitor VGA com $SW[1:0] = 00$. O resultado é visualmente equivalente ao da configuração 1.

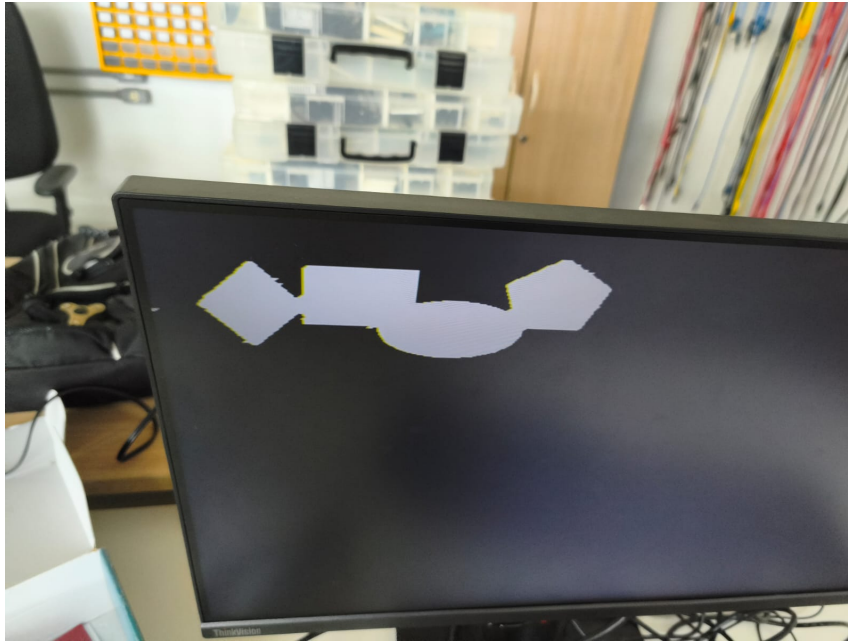


Figura 7: Imagem filtrada pela rede *mesh* com configuração 3 diagonal ativa ($n11 \rightarrow n12 \rightarrow n22 \rightarrow n23$), exibida no monitor VGA com $SW[1:0] = 00$. A equivalência com as configurações anteriores confirma a corretude do roteamento XY para diferentes topologias de caminho.

4.2 Flexibilidade de Configuração da NoC

A arquitetura proposta demonstra elevada flexibilidade: a simples alteração das constantes em `noc_controller` permite explorar diferentes caminhos de processamento sem necessidade de modificação da fiação da *mesh* ou dos nós individuais. As três configurações disponíveis (coluna vertical, grupo no topo e diagonal) produzem resultados visuais distintos, evidenciando como a topologia do caminho ativo influencia o comportamento do filtro composto.

A configuração de coluna vertical ($n10 \rightarrow n11 \rightarrow n12 \rightarrow n13$) aplica quatro estágios de filtragem em uma única coluna, maximizando a profundidade do processamento para aqueles pixels que percorrem o caminho. A configuração de grupo no topo explora nós em posições não alinhadas (formando um L), demonstrando que o roteamento XY é capaz de navegar caminhos com mudanças de direção. Em todos os casos, apenas os 4 nós do caminho realizam processamento efetivo; os demais 12 nós atuam exclusivamente como roteadores, transmitindo os dados sem modificá-los.

4.3 Clock Gating e Sincronismo com o VGA

O uso do clock gated (`clk2 = mask & pclk & on`) é fundamental para o correto funcionamento dos registradores de deslocamento internos ao `modulo_sal_e_pimenta`.

Como esses registradores constroem a janela deslizante de 3×3 pixels contando ciclos de clock, é imprescindível que o clock avance exatamente uma posição por pixel ativo da imagem. A inibição do clock fora da região ativa ($\text{mask} = 0$) e durante o *blanking* VGA ($\text{on} = 0$) garante este comportamento.

Contudo, o *clock gating* combinacional implementado por portas AND pode introduzir *glitches* no sinal de clock se os sinais de controle (mask e on) não forem estáveis durante as transições de subida do pixel clock. Em FPGA Cyclone II, recomenda-se o uso de primitivas dedicadas de *clock gating* (como ALTCLKCTRL) para evitar este problema. Para as dimensões de imagem desta prática, os efeitos práticos são toleráveis e não comprometem a funcionalidade observada.

4.4 Latência do Caminho de Processamento

Cada nó do caminho de processamento introduz uma latência de 513 ciclos de clock ativos: 256 ciclos do primeiro registrador de deslocamento, mais 256 ciclos do segundo, mais 1 ciclo do `shift_3` (para completar a janela de 3 bits da linha corrente). Com 4 nós em série, a latência total é de $4 \times 513 = 2052$ ciclos de clock ativos. A uma taxa de 256 pixels por linha e 128 linhas, a imagem completa possui 32.768 pixels ativos por quadro, de forma que a latência de 2052 pixels corresponde a aproximadamente 8 linhas de imagem. Isto significa que os primeiros 8 valores de saída do filtro são inválidos (período de *warmup*), o que pode ser observado como uma faixa escura nos primeiros pixels da imagem exibida.

4.5 Demonstração em Vídeo

O funcionamento completo do sistema — incluindo a alternância entre as três configurações de roteamento da rede *mesh*, a seleção entre imagem original, filtrada e com detecção de bordas via chaves da placa DE1, e a exibição em tempo real no monitor VGA — foi registrado em vídeo e está disponível publicamente no YouTube:

<https://www.youtube.com/shorts/twcA4ESBsw>

O vídeo demonstra a equivalência do resultado de filtragem entre as diferentes topologias de caminho configuradas no `noc_controller`, evidenciando a corretude do roteamento XY e a flexibilidade arquitetural da rede *mesh* 4×4 .

5 Conclusões

Esta prática demonstrou com sucesso a implementação de uma rede *mesh* 4×4 para processamento digital de imagens em FPGA, com roteamento determinístico XY e cadeia de processamento configurável. As principais conclusões são:

1. **A topologia *mesh* é naturalmente adequada para PDI em FPGA:** A regularidade da *mesh* facilita tanto a implementação em hardware (fiação sistemática entre vizinhos) quanto a escalabilidade (a adição de linhas ou colunas segue o mesmo padrão de interligação). O uso de parâmetros estáticos por nó (MY_ADDR) elimina lógica de endereçamento dinâmico, reduzindo o custo de área.
2. **O roteamento XY garante ausência de *deadlock* sem custo de tabelas de roteamento:** A decisão de roteamento em cada nó é puramente combinacional, baseada na comparação de 4 bits, e sintetiza-se em uma quantidade mínima de lógica. Não há estado de roteamento armazenado em nenhum nó, o que simplifica a verificação formal da correção do sistema.
3. **O *noc_controller* oferece reconfigurabilidade sem reprojetos da *mesh*:** A separação entre a estrutura da rede (*mesh_4x4*, *mesh_node*) e o caminho de processamento (*noc_controller*) permite alterar a topologia do filtro composto sem modificar a fiação da rede. Esta separação de preocupações é um princípio de projeto fundamental em arquiteturas NoC.
4. **O *clock gating* é necessário mas requer cuidado em FPGA:** A inibição do clock da rede durante períodos de *blanking* VGA é indispensável para o correto funcionamento dos registradores de deslocamento. O uso de portas AND combinacionais para *clock gating* é funcional nesta escala, mas primitivas dedicadas seriam preferíveis em projetos maiores ou com requisitos de temporização mais estritos.
5. **A latência acumulada de múltiplos estágios deve ser considerada:** O encadeamento de 4 módulos de filtro introduz uma latência de aproximadamente 8 linhas de imagem, visível nos primeiros pixels da saída filtrada. Em sistemas de processamento de vídeo em tempo real, esta latência deve ser contabilizada no projeto do pipeline de exibição.

6 Referências Bibliográficas

Referências

- [1] DALLY, William J.; TOWLES, Brian. **Principles and Practices of Interconnection Networks**. San Francisco: Morgan Kaufmann, 2004.
- [2] BENINI, Luca; DE MICHELI, Giovanni. Networks on chips: a new SoC paradigm. *IEEE Computer*, v. 35, n. 1, p. 70–78, jan. 2002.
- [3] GONZALEZ, Rafael C.; WOODS, Richard E. **Processamento Digital de Imagens**. 3. ed. São Paulo: Pearson Prentice Hall, 2010.

- [4] EMBARCADOS. **Controlador VGA — Parte 1**. Disponível em: <https://embarcados.com.br/controlador-vga-parte-1/>. Acesso em: 25 mai. 2026.
- [5] EMBARCADOS. **Controlador VGA — Parte 2**. Disponível em: <https://embarcados.com.br/controlador-vga-parte-2/>. Acesso em: 25 mai. 2026.
- [6] INTEL CORPORATION. **Quartus Prime Standard Edition Handbook**. Disponível em: <https://www.intel.com/content/www/us/en/programmable/documentation/>. Acesso em: 25 mai. 2026.
- [7] PEDRINO, Emerson Carlos. **Roteiro da Prática 7: Arquitetura Mesh para PDI**. São Carlos: UFSCar — Departamento de Computação, 2026. Disponível em: <https://ava.ufscar.br>. Acesso em: 25 mai. 2026.
- [8] PATTERSON, David A.; HENNESSY, John L. **Organização e Projeto de Computadores: a Interface Hardware/Software**. 5. ed. Rio de Janeiro: Elsevier, 2017.
- [9] THOMAS, Donald E.; MOORBY, Philip R. **The Verilog Hardware Description Language**. 5. ed. New York: Springer, 2002.